

RAI: Value Transfer with Account-Local Consensus

Anonymous Author(s)

Abstract

RAI is a value-transfer protocol for single-owner accounts with changing validator committees. It orders each account independently with single-support voting and uses joint checkpoints to change committees. Owners can submit batched sends and receives at any time, without account-level timeouts. The main challenge is preserving transfers whose final certificates appear only after a handoff begins. RAI addresses this with signed state reports and a recovery rule that preserves such late finality. Successor voting overlaps the handoff, but only checkpoint-compatible work becomes final. We prove safety under arbitrary message delays and termination under explicit availability, synchrony, and uncontested-continuation assumptions.

1 Introduction

Two payments arrive at the same time. They spend from different accounts, so neither needs to wait for the other. A transfer system should be able to confirm both without first deciding which payment came first. Single-owner accounts make this possible: each owner orders its own updates, while validators prevent conflicting spends [1–3].

Now suppose the validators change while a payment is being confirmed. Some old validators have voted for it. The recipient may already hold enough votes to show that the payment is final, even though no validator has assembled that proof. The new committee must not approve a conflicting spend. Yet waiting to hear from every old validator would let one failed validator stop the handoff.

This is the problem RAI addresses. Independent account voting is useful only if its results survive committee changes. A checkpoint must preserve not just the final certificates that its proposer has seen, but also certificates that delayed votes may form later. Preserving every disputed block as an unresolved fork is not enough: a transfer that is already final must remain final, not become one choice among several.

RAI separates two tasks. Owners propose blocks on their own account chains. Validators vote on these blocks without account-slot timers or a global proposal schedule. Periodically, the old and next committees agree on a checkpoint. That checkpoint preserves finalized history, resolves choices that the reports settle, and retains other forks without applying their operations. The next committee can begin voting while the checkpoint is being chosen; it cannot make those early votes spendable until they are checked against the decision.

Our contribution. We present a protocol that combines account-local voting with an explicit committee handoff. It

has three main parts. First, a timer-free, single-support adaptation of Kudzu’s voting rules [6] supports batched transfers and allows a finalized child to settle its unfinalized ancestors. Second, signed state reports preserve possible late finality without recording every vote a reporter has received. Third, a joint election uses two committee vote pools but one shared history, so a one-sided result cannot block reconfiguration. We give the rules and prove their safety and termination conditions.

Technical intuition. A normal final vote carries an important fact: its sender has already seen a notarization certificate for that block. Enough of these senders must appear in any report quorum. Thus a hidden normal final certificate leaves a visible notarization certificate behind. A hidden fast certificate instead leaves enough first votes. The checkpoint recovers these obligations before considering weaker conflicts. It need not guess which messages are still in flight.

Recovery after owner equivocation follows the same separation. A checkpoint may retain two unfinalized tips. The owner selects a surviving tip by extending it with a new child. Finalizing the child also finalizes its selected ancestors. This does not promise that any later owner preference will win: the child must be uncontested, including by old signed alternatives. RAI targets single-owner transfers, not general transactions that atomically update several owners’ accounts.

2 Related Work

Transfers without a global order. Guerraoui et al. [1] show that the single-owner asset-transfer object has consensus number one and give a Byzantine message-passing construction. FastPay [2] uses Byzantine consistent broadcast for prefunded payments. These results establish that preventing double spending does not require a total order of unrelated transfers. RAI does not claim to remove consensus altogether: it uses account elections and an explicit consensus protocol for checkpoints.

Account and object histories. Nano [3] gives each account its own chain and separates sending from receiving. RAI keeps that organization, but batches operations and specifies how provisional forks and final account prefixes cross committee boundaries. Sui Lutriss [4] combines broadcast for owned objects with consensus for shared objects, checkpoints, and reconfiguration. Its recovery from client equivocation makes it a close comparison. RAI instead studies account chains, report-based recovery of late certificates, and a handoff in which successor votes remain provisional until the predecessor checkpoint closes. Its uncontested-child

condition is an explicit limitation, not a claim of stronger recovery than Sui.

Fast voting and committee changes. Simplex [5] and Kudzu [6] use lightweight leader rotation. Kudzu integrates fast and normal finalization through first votes and a second-look rule. RAI keeps the fast and normal certificate thresholds, not Kudzu’s erasure-coded data path. Its account elections omit both timeout voting and second looks; its epoch election retains them and adds a shared rule for two committees. Dynamic Byzantine reliable broadcast [7] addresses asynchronous join and leave operations with delivery guarantees tied to participation. RAI solves a different problem: agreeing on a finalized checkpoint and carrying it into a pre-determined successor committee under the stated progress windows.

State transfer. A hash commitment authenticates reconstructed bytes; it does not establish that those bytes describe a valid ledger. RAI checks both. Its state differences reconstruct frozen report inventories from a shared canonical local inventory state; eventual gossip makes such a source available. The protocol-specific step is the recovery rule over signed report contents. Neither authenticated storage nor Kudzu alone supplies that rule or proves the two-committee composition.

3 Model and Guarantees

Clients own accounts and sign blocks. Validators check blocks and vote. Every committee has $N = 3f + 2p + 1$ equal-weight members, where $f, p \geq 0$ are integers. At most f members of each committee are Byzantine: they may deviate arbitrarily. Corruptions are static. Committee selection is deterministic, but ensuring the fault bound in every selected committee is a deployment assumption.

Signatures are unforgeable and hashes are collision resistant. Messages have canonical encodings and identify the network session. A certificate is a set of distinct, individually verifiable signed votes; it is not a new signed message. Correct validators retain and gossip the votes and data that justify their actions. An object available at one correct validator eventually reaches every correct validator that needs it. Retiring validators serve handoff data until correct successors hold durable copies.

Safety requires no bound on message delay. Termination requires fair processing, finite admitted work, available data, and sufficiently long periods of bounded delay [8]. During a prepared voting phase, δ bounds delivery and processing of the needed messages. State reconstruction has a separate preparation budget. Correct clocks advance at rates in $[1 - \rho, 1 + \rho]$, for $0 \leq \rho < 1$. Section 6.2 states the timer and availability conditions; neither an unavailable quorum nor unbounded overload is covered.

Accounts and checkpoints. Each account has a genesis block and a tree of owner-signed descendants. A block’s

account slot is its parent’s slot plus one. Slots are positions in an account history, not times or voting attempts. Two blocks conflict when neither is an ancestor of the other.

A closed epoch has state $S_e = (L_e, \Sigma_e)$. The ledger L_e contains finalized account prefixes and retained unfinalized branches. The application state Σ_e contains balances and the send identifiers already received; it applies finalized blocks only. A deterministic function derives committee C_e from Σ_e . Genesis supplies a finite state $S_{-1} = S_G$ and $C_{-2} = C_{-1} = C_G$.

An owner equivocates by signing conflicting blocks. A continuation is uncontested if no conflicting owner-signed alternative exists at any of its new positions, including an alternative signed earlier. Ceasing to equivocate does not revoke old signatures.

Required properties. Safety means that correct validators’ finalized histories are compatible, final blocks cannot be replaced, balances never become negative, and each send is received at most once. Termination has three parts: correct reports can be reconstructed; each epoch closes under the epoch progress conditions; and each finite valid uncontested continuation eventually finalizes and is checkpointed under the account progress conditions. Safety also covers equivocating owners. Termination does not promise a preferred outcome for a contested account position.

4 Account Voting

4.1 Blocks and validation

An account block is $B = (a, h_p, x, \sigma_a)$: an account, a parent hash, an ordered batch, and the owner’s signature. Its identifier is

$$h(B) = H(\text{Block}, a, h_p, x).$$

The epoch is deliberately absent. A block omitted at a checkpoint can be retried with fresh votes without changing its contents. The signature binds those contents in the network’s signature context. An empty batch is allowed, so an owner can extend a fork without making a payment.

An operation is $\text{Send}(b, z)$ for a positive amount z , or $\text{Receive}(s)$. A send at operation index j has identifier $s = (h(B), j)$. It debits the sender. A receive credits the named destination and must carry proof that the send is already final. A transfer is therefore complete only when its receive is final; the protocol does not make a passive recipient sign a receive.

To validate a block, replay its entire unfinalized parent prefix from a finalized anchor, then replay its batch. Check signatures, parent links, amounts, destinations, and external finality proofs. Maintain one temporary balance and one temporary received-send set throughout. Reject an overdraft or repeated receive at any point. Checking each block only against the last finalized state would miss a duplicate receive in an unfinalized parent.

This replay does not apply operations. Every finalization path repeats the check and atomically applies the selected

Table 1. Thresholds. The last column illustrates $f = p = 1$, so $N = 6$. Second looks apply only to epoch elections. These are protocol parameters, not measurements.

Evidence	Threshold	Example
Notarization (NC)	$q = N - f - p$	4
Normal finalization (FC)	q	4
Fast finalization (FF)	$N - p$	5
Second look / recovery	$r = f + p + 1$	3
Selected reports	$N - f$	5

prefix, once per operation. Finality proofs are finite and acyclic. They must verify the relevant checkpoints and account rules, not just a signature count. An external finality proof used for epoch- e account voting cannot depend on S_e or a later checkpoint.

4.2 Votes and finalization

A voting domain identifies (*session*, e , a , v). A vote also binds its type and the block hash. An identity counts once per block in a quorum, even if it sent both a first vote and a later notarization vote. Table 1 gives the thresholds.

A correct validator first-votes the first valid available block in the domain, at most once. This is its only notarization support: it issues no separate account notarization votes and never second-looks a rival. Certificate and report formats are unchanged. It records at most one first vote from each identity, including itself. A block is complete when its data, required ancestry, and NC are available. Notarization alone has no balance effect.

A validator may final-vote only for the block it first-voted, once that block is complete and eligible. Sending this vote permanently ends its voting there. It must persist that terminal state atomically with signing, not wait until it sees a final certificate. No later callback may sign in that domain.

A block finalizes when it is eligible and has either $N - p$ first votes or q final votes. The selected unfinalized ancestors finalize with it. Incompatible provisional branches are removed. Learning finality closes voting in the resolved positions. Validators still relay the underlying evidence.

4.3 Where a block may be voted on

Let $d_a(S)$ be the greatest depth of any block for account a in checkpoint S , including retained forks. Once S_{e-1} is known, new epoch- e voting starts at depth $d_a(S_{e-1}) + 1$, with a parent at that maximum depth. Further children require complete epoch- e parents. A shorter checkpoint branch is not a new starting point below this frontier.

The next epoch can open before this checkpoint is known. During that overlap, first votes, which also support notarization, may extend a maximum-depth tip of the latest closed ledger, an already-finalized parent from the closing epoch with its full proof, or a complete current-epoch parent built

Algorithm 1 Account domain (e , a , v) at a correct validator

- 1: Persist the first vote, if any, and a terminal flag.
- 2: Retain and relay validated blocks, votes, and their evidence.
- 3: Before signing, require the current open epoch, verified committee, valid placement, and a clear terminal flag.
- 4: First-vote the first valid available block once; issue no other account notarization support. Process this step before a newly enabled final vote.
- 5: Once the first-voted block is complete and eligible, atomically become terminal and final-vote for that block.
- 6: On eligible fast or normal finality, apply the checked prefix once and close its resolved domains.
- 7: At the epoch boundary, stop old-epoch signing before producing the immutable report.

in the same way. A merely notarized parent from the closing epoch is not enough. Every unresolved ancestor outside the known closed ledger must already have current-epoch certificate evidence before its descendant is voted on.

When S_{e-1} arrives, each early placement is checked again. It settles only if it starts above the decided frontier, uses the required consecutive current-epoch parents, and respects finalized history. Older unresolved ancestors must be retained in L_{e-1} . The validator keeps the exact ancestry and dependencies used for its votes, so this check does not invent missing old evidence.

A block is eligible only after its placement and entire selected prefix pass these checks. Until then, even a fast vote tally is provisional: it cannot justify a final vote, a receive, or application effects. Checkpoint arrival does not reset voting records or terminal flags. Failed early work may be discarded, but the same domain is not restarted. Epoch 0 starts directly from closed genesis.

4.4 Recovering a retained fork

Suppose a checkpoint retains competing tips A_v and B_v at maximum depth v . The owner may propose a new child A_{v+1} of A_v . Finalizing that child selects and finalizes its unresolved parent prefix, then removes the competing branch. The child must be valid and uncontested for the progress guarantee to apply. A checkpoint never creates that child for the owner, and it never reopens a retained position at depth v .

5 Epoch Handoff

An epoch handoff has three steps: freeze old account voting, reconstruct a quorum of reports, and jointly decide the checkpoint derived from those reports. Each step has a separate stopping condition.

5.1 One open epoch and one closing epoch

A validator has one open epoch and at most one closing epoch. Only the open epoch accepts new account votes. The account committee in epoch e is $K_e = C_{e-2}$. The election

Table 2. The committee lag permits voting overlap, not early application finality. The two vote pools remain distinct even when their memberships coincide.

Open epoch	Account committee	May still be closing
e	C_{e-2}	$e - 1$
$e + 1$	C_{e-1}	e
$e + 2$	C_e	$e + 1$

closing e uses old committee $O_e = C_{e-2}$ and next committee $N_e = C_{e-1}$. Its decision derives C_e , which begins account voting in epoch $e + 2$.

After a local duration D , a validator may end epoch e only if no older epoch remains closing. It stops epoch- e account signing, signs its report against closed S_{e-1} , and atomically makes e closing and $e + 1$ open. Committee $K_{e+1} = C_{e-1}$ is already known. Thus new account voting need not wait for the handoff it overlaps.

If closing e takes longer than D , the boundary of open epoch $e + 1$ waits; account voting there continues. There is never a second closing election. Learning a decided checkpoint stops any remaining voting for that epoch. Catch-up verifies checkpoints in predecessor order and does not reopen old voting domains.

5.2 Reports that remain reconstructible

Each old-committee validator signs one immutable report. It binds the session, epoch, predecessor checkpoint, committee, identity, and two roots. One root commits to its certified inventory T_i : complete, settled account blocks and any known stronger finality. The other commits to G_i : its own votes not yet summarized by that inventory. It is not a log of every vote the validator received.

An unsummarized first or notarization vote remains in G_i with its block and validating evidence. A final vote on a notarized but not yet finalized block also remains. A summarized vote must instead be covered by its corresponding certified status, or by a finalized descendant that includes it. In particular, a normal final voter retains the notarization evidence it had before signing. An early vote that fails settlement remains historical evidence but contributes no checkpoint candidate or recovery count.

Reports include the finite closure of required ancestry and external finality evidence. Their roots commit to canonical semantic records and references, not an arbitrary ordering or subset of signatures that witnesses the same status. The records fix which certificate claims are represented; fetching a different equivalent witness cannot add a new status. All claimed statuses are independently verified. A Byzantine reporter may omit facts, but cannot make missing signatures or invalid blocks valid.

A report freezes these historical objects, not the validator’s live knowledge. Late votes may form new certificates, and

successor voting may continue in a separate domain. Neither changes the signed report. Correct validators retain the historical contents, including evidence for branches pruned from their live view.

Reconstruction. A requester with state root r_s asks for the difference to signed target r_t . A responder knowing both states returns canonical edits and authentication hashes. The requester reconstructs the target, recomputes its root, and separately verifies its ledger semantics. Edits may add, remove, or change the status of records. Root equality alone is not a validity proof.

Correct validators continuously update their canonical local inventories and roots as historical records arrive. By eventual gossip and finite admitted work, the inventories eventually contain the same records and certified statuses and therefore have the same root. Fair retry then provides a source root shared by requester and a correct reporter, from which the reporter’s frozen inventory is reconstructed. The requester also fetches the residual object and all referenced evidence. An unavailable Byzantine report is simply not usable. Any validator voting on a proposal retains and serves its reconstructed inputs.

5.3 Building a checkpoint

An epoch proposal selects exactly $N - f$ usable reports from distinct old-committee identities. Let Q be their canonically ordered signed identifiers and roots. All validators compute

$$S_e = \text{BuildState}(S_{e-1}, Q), \quad d_e = H(S_e).$$

The proposal carries Q and d_e , not a serialized checkpoint. Once selected, its inputs do not change with a validator’s private view. Construction is a pure computation on a fresh copy of the predecessor state: validating a proposal changes no live balances. Only the joint decision installs its checkpoint. Installation reconciles already-finalized live operations without applying their effects twice.

The construction starts with the entire predecessor ledger. It adds finalized evidence and valid new candidates exposed by the selected reports, together with their required ancestry. A new candidate needs either a represented settled notarization certificate or one selected reporter’s own residual first/notarization vote. All new candidates satisfy the placement and branch checks; only a residual candidate may lack its own notarization certificate. An old unresolved block must already be in the predecessor ledger.

Recover before choosing. The checkpoint first preserves all known finality. It then applies two recovery rules at each new account position, using the full selected evidence before pruning:

1. If exactly one block has a represented notarization certificate, finalize it and its selected ancestors.

2. If no block has such a certificate and exactly one block has at least $r = f + p + 1$ selected reporters' own residual first votes, finalize that block and its selected ancestors. These are checkpoint decisions, not claims that an account final certificate is already available. The first rule recovers hidden normal finality; the second covers hidden fast finality. A single residual conflict cannot defeat a unique full certificate. Votes merely received from someone else do not count as a reporter's own first vote.

For example, let $f = p = 1$, so there are six validators and a report quorum contains five. A four-vote normal final certificate intersects those reports in at least two correct final voters. Each already had the notarization certificate. A five-first-vote fast certificate leaves at least three correct first voters in the selection; each contributes its first vote or stronger certified evidence. This is enough to protect the block even if the final certificate appears later.

Resolve what remains. After mandatory recovery, discard branches that conflict with finalized history. If only one block remains at an account position, finalize it and its selected ancestors. Apply this rule in simultaneous rounds until no change occurs. Keep the other branches as checkpoint-notarized: the checkpoint preserves them without applying their operations. This status does not assert that each block has an account NC.

Inherited provisional forks are retained until a finalized branch excludes them. Unselected current-epoch work may be omitted, leaving a clean position for a later epoch. Each finalized operation is applied once, in a canonical order that respects account parents and finalized-send dependencies. Within a block, operations retain their signed order. A lexicographic tie-break orders independent blocks; it has no online ordering role.

5.4 One election, two committee vote pools

The epoch election has numbered slots $w = 1, 2, \dots$, distinct from account slots. Each slot has a leader chosen in round-robin order from the union of the two committees. Both committees validate the same proposal and shared parent history, but sign in separate committee domains. Even an identity in both committees has two vote records.

A placement contains its slot, parent hash, and payload (Q, d_e) . Its leader signs the placement. Election genesis is at slot 0. A genesis child may introduce any valid report selection. A non-genesis child must copy its parent's payload. Each parent must be in a strictly earlier slot and have matching notarization certificates in both committees. Skipped intervening slots require the shared evidence defined below. Before any non-timeout vote, validators reconstruct the reports, run BuildState, and verify this parent proof.

Voting inside one committee. A validator first-votes the first valid available leader proposal. If none is available before local timeout Δ_E , it first-votes a special timeout value. As in

Algorithm 2 BuildState(S_{e-1}, Q)

- 1: Reconstruct and verify the predecessor and all selected reports; ignore well-formed but unsettled early votes.
 - 2: Form the parent-closed candidate forest, including inherited forks and required final dependencies.
 - 3: Compute mandatory targets from final evidence and both recovery rules, before pruning.
 - 4: Check their parent closure for compatibility and validity; mark it final and remove conflicting branches.
 - 5: In simultaneous rounds, finalize sole survivors and their ancestors; remove conflicting branches.
 - 6: Retain unresolved survivors without application effects. Replay newly final operations once into the candidate state, in dependency order.
 - 7: Return the canonical ledger and application state. Reject malformed evidence or an invalid dependency graph.
-

account voting, a first vote also supports notarization. Unlike account voting, after first-voting, a block with r recorded first votes may receive second-look support once its data and proof are valid. A validator may also support timeout when the total recorded first votes minus the largest count for any non-timeout block is at least r . Count each sender once; use zero when no non-timeout first vote is recorded.

On obtaining a complete local NC, the validator ends voting in that committee's slot. It sends a final vote exactly when it has supported no other value, including timeout. An available first-vote step precedes this terminal transition. A timeout certificate, denoted TC, also ends local voting. A fast tally alone does not suppress the normal-vote transition. All data and constituent votes continue to be relayed after termination.

Shared progress. Matching $NC_O(h)$ and $NC_N(h)$ make a placement jointly complete. It decides the checkpoint only with matching fast certificates in both committees or matching normal final certificates in both. A mixed fast/normal pair alone is not a decision; the normal path remains available. No one-sided certificate is an epoch decision or an independent history lock.

A slot may instead be skipped on

$$\text{Skip}(w) \iff \text{TC}_O(w) \vee \text{TC}_N(w) \vee \exists h \neq h' : \text{NC}_O(w, h) \wedge \text{NC}_N(w, h'). \quad (1)$$

Thus one committee finalizing h while the other times out causes a shared skip, not a deadlock. A validator advances only after joint completeness or shared skip evidence. It relays that evidence and the parent/skip chain, allowing others to catch up. A shared skip may also terminate unfinished local voting.

A correct leader chooses a highest available joint-complete parent for which all intervening slots have skip evidence, breaking ties canonically. It copies that parent's payload, or selects reports when extending genesis. Its fresh-selection pool keeps the first fully usable report from each identity;

later equivocating reports do not replace that representative. Other leaders' correctly signed usable selections can still be validated. This bounds each correct leader's selection pool without giving a private residual vote a veto over the election.

6 Protocol Analysis

We first prove agreement inside an election, then show that reports preserve its result across handoffs. Cryptographic statements hold except with negligible signature-forgery or hash-collision probability. The accompanying proof supplement expands the arguments; the rules and proof steps needed for the guarantees are given here.

6.1 Safety

Lemma 6.1 (Account NC uniqueness and finality exclusion). *In one account domain, at most one block has an NC. In either an account domain or an epoch-election committee, $FC(h)$ or $FF(h)$ excludes $NC(h')$ for $h' \neq h$, including timeout in an epoch election.*

Proof. Two size- q quorums intersect in at least $2q - N = f + 1$ identities, including a correct one. In an account domain, that identity cannot support two blocks, so two conflicting NCs are impossible. Both account finality forms imply an NC for the finalized block: normal final voters require it, and $N - p \geq q$ first votes supply it. Account finality exclusion follows.

Inside an epoch-election committee, a normal final voter has never supported a conflict and cannot do so later. The same quorum intersection excludes a conflicting notarization.

For fast exclusion, let $f' \leq f$ be the actual number of Byzantine members. At least $N - p - f'$ correct members first-voted for h , leaving at most p correct first voters for everything else. In either domain, a different block has at most $f' + p \leq f + p < r$ first supporters. In the epoch election, it therefore cannot trigger a correct second look or reach q by first votes alone. The split-first timeout test also stays below r . All correct first voters validated the fast-supported proposal, so it is not an invalid-data case. $\square$

Lemma 6.2 (Joint agreement). *A jointly finalized epoch placement excludes a skip at its slot. All joint decisions for that epoch select the same checkpoint.*

Proof. Joint finality supplies final evidence in both committees. Lemma 6.1 excludes either timeout certificate. For conflicting cross-committee notarizations, any proposed joint-finalized value differs from at least one of them, so that committee excludes its finality. Thus every clause of (1) is impossible.

A joint-complete placement at a greater slot cannot skip the finalized slot. Following its strictly decreasing parent slots must reach a joint-complete placement there, and

Lemma 6.1 makes it the finalized one. Descendants copy its payload. Two decisions in the same slot also cannot conflict. The payload and deterministic construction therefore identify one checkpoint. No step assumes independent committee histories. $\square$

Lemma 6.3 (Deterministic checkpoint construction). *For a valid predecessor and usable selected reports, the mandatory targets are compatible. BuildState terminates with one output.*

Proof. Settled current-epoch account NCs have compatible selected prefixes. Equal-depth conflicts violate NC uniqueness; a deeper conflicting tip would require a second NC at the shallower tip's depth. Both contradict Lemma 6.1. This also covers eligible account finalizations and their ancestors.

Consider a recovery target. A deeper conflicting target must expose a current-epoch notarization at the shallower position, because voting on descendants requires complete parents. This remains true after overlap: all unresolved ancestors above the final checkpoint frontier already needed current-epoch certificates. Such a certificate contradicts either uniqueness of the shallower certified target or the absence of certificates required for residual-first recovery. A residual-first recovery target has at least one correct reporter, since $r > f$, so its required ancestry is available. At equal depths the singleton tests exclude two recovery targets; final evidence excludes a conflicting certificate by Lemma 6.1. Inherited finality is respected by the placement rules.

After this compatible mandatory closure, the candidate forest is parent-closed. A deeper block conflicting with a sole survivor would supply a second block at the survivor's depth. Thus each simultaneous round is compatible. A non-terminal round finalizes or removes at least one of finitely many blocks. Canonical inputs, simultaneous rounds, and the specified replay order give one output. $\square$

Lemma 6.4 (Reports preserve even late finality). *If an eligible epoch- e account final certificate can be formed, every valid checkpoint built from $N - f$ reports finalizes its block and selected ancestors. The certificate may be assembled after reporting or after the checkpoint decision.*

Proof. Let Q be the selected reporter identities. A size- q normal final signer set F has at least

$$|F \cap Q| - f \geq q - f - f = p + 1$$

correct members in the selection. They final-voted before their own reports: correct reporters cannot sign old account votes afterward. Each had the block's NC and reports that status or stronger finality covering it. No conflicting NC exists by Lemma 6.1. The unique-certificate recovery rule therefore finalizes the block.

A fast signer set A leaves at least

$$|A \cap Q| - f \geq (N - p) - 2f = f + p + 1 = r$$

correct first voters in the selection. If one reports a summarizing certificate, the first case applies. Otherwise their own first votes remain in the residual objects. By the first-vote count in Lemma 6.1, at most $f + p < r$ first supporters exist for every different block in the domain, so only this block can reach the recovery threshold. The second rule applies.

Settlement cannot discard these supports for an eligible block: correct voters retain the ancestry required before signing, and reporting uses the fixed decided predecessor. Withheld Byzantine signatures and later votes from unselected members do not change either intersection. The conclusion concerns possible certificates, not just certificates already observed. $\square$

Theorem 6.5 (Transfer safety). *Finalized account histories are compatible and irreversible across all epochs. Finalized balances are nonnegative, and each send is received at most once.*

Proof. Induct from genesis. Lemma 6.2 gives one checkpoint per epoch. Within an epoch, Lemma 6.1 excludes conflicting finalized tips and their selected prefixes. Lemma 6.4 preserves all of them at the boundary, including late certificates; Lemma 6.3 makes additional checkpoint choices compatible.

Successor votes cannot break this induction. Before the predecessor is decided, they authorize no final votes or application effects. Afterward, eligibility places them above its maximum represented frontier and requires compatibility with its finalized prefix. Every later checkpoint preserves that prefix. No vote record is reset during settlement.

Finally, induct over finalization events. Each event replays the entire selected prefix from its finalized anchor and checks every intermediate balance and receive identifier. A receive refers to an irreversible send addressed to that account. One compatible destination history cannot receive it twice. The applied-operation record prevents checkpoint installation from applying an already-finalized live operation again. $\square$

6.2 Termination

Lemma 6.6 (Report reconstruction terminates). *A correct reporter's report becomes usable under retained-data availability, fair retry and service, and eventual message delivery.*

Proof. By §5.2, finite admitted work and eventual gossip make correct validators' continuously updated local inventory roots converge. A correct reporter retains its signed target. Fair retry therefore eventually uses the shared local root to reconstruct that target, then fetches the residual votes and finite evidence closure. Root checks and semantic validation finish. An unavailable Byzantine report is unnecessary because $N - f$ correct reports suffice. Lemma 6.3 then gives finite local checkpoint computation. $\square$

Lemma 6.7 (An epoch-election slot advances). *In a sufficiently long prepared synchronous interval with at least $N - f$*

responsive correct validators in each committee, a shared slot yields joint completeness or a shared skip.

Proof. Consider one committee. If a correct validator has terminated, it relays the certificate causing termination, or the already sufficient shared skip. Otherwise all responsive correct validators eventually first-vote, using timeout when needed. If some non-timeout value has r correct first voters, its valid data are available and every still-voting correct validator can second-look it, producing NC. If none does, the correct votes alone satisfy

$$all - max \geq (N - f) - (r - 1) = r.$$

Adding Byzantine first votes cannot decrease this difference. The timeout rule produces TC. Thus each committee supplies a non-timeout certificate or a timeout. A timeout on either side gives a shared skip. Otherwise matching certificates give joint completeness and different ones give the cross-committee skip. $\square$

Theorem 6.8 (Epoch termination). *Suppose correct reports remain available, each committee has at least $N - f$ responsive correct members during progress windows, and processing is fair. Suppose recurring synchronous windows cover catch-up, report preparation, a full leader cycle, and voting. With slot-entry skew bounded by κ_E and proposal preparation and validation bounded by τ_E in those windows, require*

$$\frac{\Delta_E}{1 + \rho} > \kappa_E + \tau_E + \delta. \quad (2)$$

Then every enabled epoch handoff eventually closes.

Proof. Enough reports can be reconstructed. The checkpoint construction then gives a valid proposal. Selected data remain immutable; Lemma 6.4 prevents late account finality from invalidating their result. Provisional successor votes add no private veto.

Lemma 6.7 moves past stalled slots. Round-robin leadership over the known committee union has a bounded gap between correct leaders. In a fresh correct-leader slot within a progress window, (2) makes its valid proposal arrive before the earliest first-vote timeout. All correct first votes support it. Byzantine votes alone cannot trigger a conflicting second look or split timeout. Each committee obtains NC and then q terminal final votes. Their joint finality closes the epoch. The single-closing-epoch rule then enables a deferred boundary. Induction gives ordered closure. $\square$

The preparation and entry bounds are resource assumptions, not zero-cost state transfer. A deployment must bound admitted work and choose timeouts to satisfy them. Synchronous periods and correct leaders occurring separately, without a long enough overlap, do not imply termination.

Theorem 6.9 (Account termination and recovery). *Assume epoch termination. Let an owner persistently submit a finite valid uncontested continuation of a finalized frontier, or a*

valid fresh uncontested child of a retained maximum-depth tip. Suppose its final send dependencies become available and recurring common-open synchronous intervals contain enough time after predecessor closure to process its next block. Then every block in the continuation eventually finalizes and is checkpointed.

Proof. In a ready interval, every correct validator either already first-voted the uncontested next block or can do so now. No competing owner-signed block can consume its first vote. At least $N - f \geq q$ correct first votes give notarization. Those same validators can final-vote for that block, giving normal finalization. There is no account-slot timeout. A boundary may interrupt an attempt, but report recovery preserves any completed finality. A represented sole candidate can also finalize at the checkpoint; an omitted candidate can be retried with fresh epoch votes. The ready-interval premise supplies an attempt with enough time to finish.

Finalizing a fresh child also selects its retained ancestor prefix. Apply the argument successively to each block of the finite continuation. Lemma 6.4 and epoch termination eventually checkpoint every result. $\square$

With prepared ancestry and dependencies, a common-open interval lasting 3δ after broadcast suffices for the normal path; $N - p$ timely correct first voters give the fast path in 2δ . These are communication bounds, not measurements. As one sufficient timer budget, if openings differ by at most κ and readiness takes at most R after the last opening, then

$$\frac{D}{1 + \rho} > \kappa + R + 3\delta$$

leaves room for normal finality before the earliest boundary. A deferred boundary can only lengthen that window. Merely stopping new equivocation is not sufficient: an old conflicting signature can still defeat the uncontested premise.

6.3 Costs and limits

A correct validator signs at most one first vote and one normal final vote, for the same block, per account domain. There are no account second looks. This bounds local signing, not total network traffic: gossip, full blocks, certificate witnesses, and retained histories still have costs. RAI does not inherit Kudzu’s balanced erasure-coded communication bound.

Early successor voting can save waiting, but a checkpoint can invalidate that work. Split first votes may remain unresolved until checkpoint recovery or an uncontested child; no account second look repairs the split. Fork retention can grow storage until an owner supplies a resolving child. Historical reconstruction can require the full state. Independent account voting does not provide resource isolation or an unlimited-load guarantee. Committee admission, adaptive corruption, and automatic receive generation are outside the model.

7 Implementation and Evaluation

The implementation and experiments remain pending. The prototype is planned on the RsNano codebase [9]. No measured performance or implementation result is claimed.

8 Conclusion

RAI lets owners advance separate account histories while committees change. Its handoff preserves the consequences of votes, including final certificates that arrive late, rather than only the certificates a checkpoint proposer has seen. Provisional successor voting overlaps that handoff without making provisional work spendable. The result is a protocol with irreversible account finality and explicit, separate termination guarantees for account continuations, state reconstruction, and epoch closure.

Use of AI tools. ChatGPT assisted with rewriting, protocol review, proof exposition, and LaTeX preparation. No RAI implementation or experimental performance measurements were generated.

References

- [1] Rachid Guerraoui, Petr Kuznetsov, Matteo Monti, Matej Pavlovic, and Dragos-Adrian Seredinschi. 2019. The Consensus Number of a Cryptocurrency. In *PODC*. 307–316. <https://doi.org/10.1145/3293611.3331589>.
- [2] Mathieu Baudet, George Danezis, and Alberto Sonnino. 2020. FastPay: High-Performance Byzantine Fault Tolerant Settlement. In *AFT*. <https://doi.org/10.1145/3419614.3423249>.
- [3] Colin LeMahieu. n.d. Nano: A Feeless Distributed Cryptocurrency Network. White paper. https://content.nano.org/whitepaper/Nano_Whitepaper_en.pdf.
- [4] Sam Blackshear, Andrey Chursin, George Danezis, Anastasios Kichidis, Lefteris Kokoris-Kogias, Xun Li, Mark Logan, Ashok Menon, Todd Nowacki, Alberto Sonnino, Brandon Williams, and Lu Zhang. 2024. Sui Lutris: A Blockchain Combining Broadcast and Consensus. In *CCS*. 2606–2620. <https://doi.org/10.1145/3658644.3670286>.
- [5] Victor Shoup. 2024. Sing a Song of Simplex. In *DISC, LIPIcs* 319. 37:1–37:22. <https://doi.org/10.4230/LIPIcs.DISC.2024.37>.
- [6] Victor Shoup, Jakub Sliwinski, and Yann Vonlanthen. 2025. Kudzu: Fast and Simple High-Throughput BFT. arXiv:2505.08771v2. September 29, 2025. <https://doi.org/10.48550/arXiv.2505.08771>.
- [7] Rachid Guerraoui, Jovan Komatovic, Petr Kuznetsov, Yvonne-Anne Pignolet, Dragos-Adrian Seredinschi, and Andrei Tonkikh. 2020. Dynamic Byzantine Reliable Broadcast. Extended technical report. arXiv:2001.06271v2. <https://doi.org/10.48550/arXiv.2001.06271>.
- [8] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. 1988. Consensus in the Presence of Partial Synchrony. *J. ACM* 35, 2, 288–323. <https://doi.org/10.1145/42282.42283>.
- [9] RsNano contributors. n.d. RsNano. Software repository. <https://github.com/rsnano-node/rsnano-node>. Accessed September 22, 2026. The experimental revision remains to be specified.